

SAKARYA ÜNİVERSİTESİ

Bilgisayar ve Bilişim Bilimleri Fakültesi
Bilgisayar Mühendisliği Bölümü

SİSTEM PROGRAMLAMA 2025-2026 BAHAR DÖNEMİ PROJESİ

tarsau - Arşivleme Programı

Hazırlayan: Baran Yeşilyurt

Öğrenci No: G231210078

GitHub: <https://github.com/YesilBaraN/tarsau>

Mayıs 2026

İçindekiler

1. Giriş
2. Programın Genel Yapısı
3. Geliştirme Süreci
 - a. Arşiv Dosya Formatı (.sau)
 - b. Metin Dosyası Kontrolü
 - c. Arşiv Oluşturma (-b)
 - d. Arşiv Açma (-a)
4. Kod Açıklamaları
5. Derleme ve Çalıştırma
6. Test Sonuçları ve Ekran Çıktıları
7. Sonuç

1. Giriş

Bu projede, Linux ortamında C dilinde çalışan **tarsau** adında bir arşivleme programı geliştirilmiştir. Program, tar, rar veya zip benzeri şekilde çalışmakta ancak herhangi bir sıkıştırma işlemi yapmamaktadır. Temel olarak birden fazla metin dosyasını tek bir **.sau** uzantılı arşiv dosyasında birleştirmekte ve bu arşiv dosyasından tekrar orijinal dosyaları çıkartabilmektedir.

Program komut satırından iki farklı modda çalışmaktadır:

- **tarsau -b dosya1 dosya2 ... -o arxiv.sau** → Dosyaları birleştirip arşiv oluşturma
- **tarsau -a arxiv.sau [dizin]** → Arşiv dosyasını açma

2. Programın Genel Yapısı

Proje tek bir C kaynak dosyasından (**tarsau.c**) ve bir **Makefile** dosyasından oluşmaktadır. Program içerisinde kullanılan temel fonksiyonlar ve görevleri aşağıdaki tabloda özetlenmiştir:

Fonksiyon	Görevi
<code>metin_mi()</code>	Bir dosyanın ASCII metin dosyası olup olmadığını kontrol eder
<code>dosya_boyu()</code>	<code>stat()</code> ile dosyanın boyutunu bayt cinsinden döndürür
<code>izinleri_al()</code>	Dosyanın izinlerini octal string olarak okur (ör: "0644")
<code>arsiv_olustur()</code>	-b modunda dosyaları birleştirip .sau arşivi oluşturur

arsiv_ac()	-a modunda arşiv dosyasını açıp dosyaları çıkartır
kullanım_bilgisi()	Yanlış kullanımda yardım mesajı gösterir
main()	Komut satırı argümanlarını ayrıştırır ve ilgili fonksiyonu çağırır

Kullanılan kütüphaneler:

```
#include <stdio.h>      // dosya işlemleri (fopen, fread, fprintf vs.)
#include <stdlib.h>     // genel (malloc, free, atoi)
#include <string.h>     // string işlemleri (strcmp, strcpy, strcat)
#include <sys/stat.h>   // dosya bilgileri (stat)
#include <sys/types.h>  // veri tipleri
#include <unistd.h>     // POSIX (access, chmod)
#include <errno.h>      // hata kodları
```

3. Geliştirme Süreci

a) Arşiv Dosya Formatı (.sau)

Arşiv dosyası iki ana bölümden oluşmaktadır. İlk bölüm organizasyon (içindekiler) bilgilerini, ikinci bölüm ise arşivlenmiş dosyaların içeriklerini barındırmaktadır. Format şu şekildedir:

```
[10 bayt: org bölüm boyutu][dosya1_adi,izin,boyut|dosya2_adi,izin,boyut|...][dosya1_i
```

İlk 10 bayt, organizasyon bölümünün ASCII formatındaki sayısal boyutunu tutar. Bu sayede arşiv açılırken önce bu 10 bayt okunarak organizasyon bölümünün ne kadar yer kapladığı anlaşılır. Her dosya kaydı |dosya_adi,izinler,boyut| formatında pipe karakterleriyle çevrelenir. Dosya içerikleri ise organizasyon bölümünün hemen ardından herhangi bir ayırıcı olmadan art arda yerleştirilir.

Örnek bir arşiv dosyasının iç yapısı:

```
0000000030|t1,0755,6||t2,0644,7|merhababurada!
|_____| |_____| |_____|
10 bayt   org bölümü (30b)   dosya içerikleri
```

b) Metin Dosyası Kontrolü

Programın sadece metin dosyalarını arşivlemesi gerektiği için, her giriş dosyası öncelikle ASCII kontrolünden geçirilir. Dosyadaki her bayt tek tek okunur ve 0-127 aralığında olup olmadığı kontrol edilir. Ayrıca tab, newline ve carriage return dışındaki kontrol karakterleri de reddedilir. Bu sayede binary dosyalar arşivlenmez.

```
int metin_mi(const char *yol)
{
    FILE *f = fopen(yol, "rb");
    if (f == NULL)
        return 0;

    int c;
    while ((c = fgetc(f)) != EOF) {
        if (c > 127) {          /* ascii disinda */
            fclose(f);
            return 0;
        }
        /* tab newline cr disindaki kontrol karakterlerini reddet */
        if (c < 32 && c != '\t' && c != '\n' && c != '\r') {
            fclose(f);
            return 0;
        }
    }
    fclose(f);
    return 1;
}
```

c) Arşiv Oluşturma (-b)

arsiv_olustur() fonksiyonu aşağıdaki adımları sırasıyla gerçekleştirir:

1. Dosya sayısının 32'yi aşmadığı kontrol edilir
2. Her dosyanın var olduğu ve okunabildiği doğrulanır
3. Her dosyanın metin dosyası olup olmadığı metin_mi() ile kontrol edilir
4. Toplam boyutun 200 MB'ı geçmediği hesaplanır
5. Her dosya için |dosya_adi,izin,boyut| formatında organizasyon kaydı oluşturulur
6. Çıktı dosyasına sırasıyla 10 baytlık başlık, organizasyon bölümü ve dosya içerikleri yazılır

Fonksiyonun organizasyon bölümünü oluşturan kısmı:

```
/* organizasyon bolumunu hazirla */
char org[8192];
org[0] = '\0';

for (i = 0; i < adet; i++) {
    char izin[8];
    izinleri_al(liste[i], izin);
    long b = dosya_boyu(liste[i]);

    char parca[512];
    sprintf(parca, "|%s,%s,%ld|", liste[i], izin, b);
    strcat(org, parca);
}

int org_boy = (int)strlen(org);
```

Dosyaya yazma işlemi:

```
/* baslik: 10 baytlik org boyutu */
fprintf(af, "%010d", org_boy);

/* organizasyon kayitlari */
fputs(org, af);

/* dosya iceriklerini siraya koy */
for (i = 0; i < adet; i++) {
    FILE *gf = fopen(liste[i], "r");
    int ch;
    while ((ch = fgetc(gf)) != EOF)
        fputc(ch, af);
    fclose(gf);
}
```

d) Arşiv Açma (-a)

arsiv_ac() fonksiyonu tam tersi işlemi yapar:

1. Arşiv dosyasının ilk 10 baytı okunarak organizasyon bölümünün boyutu öğrenilir
2. Organizasyon bölümü okunup her kayıt parse edilerek dosya adı, izin ve boyut bilgileri çıkartılır
3. Hedef dizin belirtilmişse oluşturulur
4. Her dosya için boyut bilgisi kadar veri okunarak hedef dizine yazılır
5. chmod() ile orijinal dosya izinleri geri yüklenir

Kayıt ayrıştırma ve dosya çıkartma kısmı:

```
/* alanlari ayir: ad,izin,boyut */
char ad[256], izin_s[16];
long boy;
sscanf(kayit, "%[^,],%[^,],%ld", ad, izin_s, &boy);

/* icerigi oku */
char *veri = (char *)malloc(boy + 1);
fread(veri, 1, boy, af);

/* dosyayi diske yaz */
FILE *df = fopen(yol, "w");
fwrite(veri, 1, boy, df);
fclose(df);

/* izinleri geri yukle */
int mod = (int)strtol(izin_s, NULL, 8);
chmod(yol, mod);
```

4. Kod Açıklamaları

Sabitler

Programın başında tanımlanan sabitler, ödevin gereksinimlerine göre belirlenmiştir:

```
#define EN_FAZLA_DOSYA    32                // max dosya sayisi
#define BOYUT_SINIRI      (200 * 1024 * 1024) // 200 MB
#define BASLIK_UZUNLUK    10                // org boyut alanı
#define ONAYLANMIS_UZANTI ".sau"            // arşiv uzantisi
#define STANDART_CIKTI    "a.sau"           // varsayılan çıktı
```

İzin Okuma

Dosya izinleri `stat()` sistem çağrısı ile okunmaktadır. `st_mode` alanından sadece izin bitleri (& 0777) alınarak octal formatta string olarak saklanır. Bu format sayesinde arşiv açılırken `strtol()` ile tekrar sayısal değere çevrilip `chmod()` fonksiyonuna verilir.

```
void izinleri_al(const char *yol, char *sonuc)
{
    struct stat bilgi;
    if (stat(yol, &bilgi) != 0) {
        strcpy(sonuc, "0644");
        return;
    }
    sprintf(sonuc, "%04o", bilgi.st_mode & 0777);
}
```

main() Fonksiyonu ve Argüman Ayırıştırma

`main()` fonksiyonu `argv[1]` parametresine bakarak hangi modda çalışılacağını belirler. `-b` modunda giriş dosyaları ve `-o` parametresi ayrıştırılır, `-a` modunda ise arşiv dosyası adı ve opsiyonel izin parametresi alınır.

5. Derleme ve Çalıştırma

Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11
HEDEF = tarsau

all: $(HEDEF)

$(HEDEF): tarsau.c
    $(CC) $(CFLAGS) -o $(HEDEF) tarsau.c

clean:
    rm -f $(HEDEF)
```

Derleme

```
baran_yesilyurt@Baran:~/tarsau$ make
gcc -Wall -Wextra -std=c11 -o tarsau tarsau.c
```

Şekil 1: Programın make ile derlenmesi (hata ve uyarı olmadan)

6. Test Sonuçları ve Ekran Çıktıları

Test 1: Temel Arşiv Oluşturma ve Açma

Üç adet test dosyası oluşturulup farklı izinlerle arşivlenmiş ve ardından açılmıştır:

```
$ echo "merhaba bu birinci dosya" > t1
$ echo "ikinci dosyanin icerigi burada" > t2
$ echo "ucuncu test dosyasi" > t3.txt
$ chmod 755 t1
$ chmod 644 t2
$ chmod 600 t3.txt

$ ./tarsau -b t1 t2 t3.txt -o deneme.sau
Dosyalar birlestirildi.

$ ./tarsau -a deneme.sau cikarilan
cikarilan dizininde t1, t2 ve t3.txt dosyalari acildi.
```

Şekil 2: Arşiv oluşturma ve açma işlemi

İçerik Doğrulama

```
$ diff t1 cikarilan/t1 && echo "t1 OK"
t1 OK
$ diff t2 cikarilan/t2 && echo "t2 OK"
t2 OK
$ diff t3.txt cikarilan/t3.txt && echo "t3.txt OK"
t3.txt OK
```

Şekil 3: Açılan dosyaların orijinalleriyle karşılaştırılması

İzin Doğrulama

```
$ ls -la cikarilan/
total 20
drwxr-xr-x 2 baran_yesilyurt baran_yesilyurt 4096 May 21 01:19 .
drwxr-xr-x 3 baran_yesilyurt baran_yesilyurt 4096 May 21 01:19 ..
-rwxr-xr-x 1 baran_yesilyurt baran_yesilyurt 25 May 21 01:19 t1
-rw-r--r-- 1 baran_yesilyurt baran_yesilyurt 31 May 21 01:19 t2
-rw----- 1 baran_yesilyurt baran_yesilyurt 20 May 21 01:19 t3.txt
```

Şekil 4: Dosya izinlerinin orijinaliyle aynı olduğunun doğrulanması (t1:755, t2:644, t3.txt:600)

Test 2: Varsayılan Çıktı Dosyası

```
$ ./tarsau -b d1
Dosyalar birlestirildi.
```

```
$ ls a.sau
a.sau
```

Şekil 5: -o belirtilmediğinde varsayılan olarak a.sau oluşturulması

Test 3: Var Olmayan Dosya Hatası

```
$ ./tarsau -b olmayan_dosya.txt -o hata.sau
olmayan_dosya.txt giris dosyasinin formati uyumsuzdur!
```

Şekil 6: Var olmayan dosya için hata mesajı

Test 4: Binary Dosya Reddi

```
$ printf '\x00\x01\x02\x03' > binary_dosya
$ ./tarsau -b binary_dosya -o hata.sau
binary_dosya giris dosyasinin formati uyumsuzdur!
```

Şekil 7: Binary (metin olmayan) dosyanın reddedilmesi

Test 5: Bozuk Arşiv Dosyası

```
$ echo "bozuk icerik" > bozuk.sau
$ ./tarsau -a bozuk.sau cikti
Arsiv dosyasi uygunsuz veya bozuk!
```

Şekil 8: Bozuk arşiv dosyası için hata mesajı

Test 6: Yanlış Uzantılı Dosya

```
$ ./tarsau -a yanlis.tar cikti
Arsiv dosyasi uygunsuz veya bozuk!
```

Şekil 9: .sau uzantısı olmayan dosyanın reddedilmesi

Test 7: Parametresiz ve Hatalı Parametre

```
$ ./tarsau
Kullanım:
  tarsau -b dosya1 dosya2 ... [-o arshiv.sau]
  tarsau -a arshiv.sau [hedef_dizin]

$ ./tarsau -x
Bilinmeyen parametre: -x
Kullanım:
  tarsau -b dosya1 dosya2 ... [-o arshiv.sau]
  tarsau -a arshiv.sau [hedef_dizin]
```

Şekil 10: Parametresiz ve geçersiz parametrelili çalıştırma

Test 8: Çok Dosyalı Arşiv (5 dosya)

```
$ ./tarsau -b coklu_1.txt coklu_2.txt coklu_3.txt coklu_4.txt coklu_5.txt -o  
coklu.sau  
Dosyalar birlestirildi.  
$ ./tarsau -a coklu.sau coklu_cikti  
coklu_cikti dizininde coklu_1.txt, coklu_2.txt, coklu_3.txt, coklu_4.txt ve  
coklu_5.txt dosyalari acildi.
```

Şekil 11: 5 dosyalı arşiv oluşturma ve açma

Test 9: Dizin Belirtmeden Açma

```
$ ./tarsau -a dizinsiz.sau  
Mevcut dizinde dtest dosyalari acildi.
```

Şekil 12: Dizin parametresi verilmediğinde mevcut dizine açılma

Test Sonuçları Özet Tablosu

#	Test Senaryosu	Beklenen Davranış	Sonuç
1	Temel arşivleme ve açma	Dosyalar doğru birleştirilip açılmalı	✓ Başarılı
2	İzinlerin korunması	755, 644, 600 izinleri aynen kalmalı	✓ Başarılı
3	Varsayılan çıktı (a.sau)	-o verilmezse a.sau oluşturulmalı	✓ Başarılı
4	Var olmayan dosya	Hata mesajı ve temiz çıkış	✓ Başarılı
5	Binary dosya reddi	Format uyumsuz mesajı	✓ Başarılı
6	Bozuk arşiv dosyası	Uygunsuz veya bozuk mesajı	✓ Başarılı
7	Yanlış uzantı	Uygunsuz mesajı	✓ Başarılı
8	Parametresiz çalıştırma	Kullanım bilgisi	✓ Başarılı
9	Çok dosyalı arşiv (5 dosya)	Tüm dosyalar doğru açılmalı	✓ Başarılı
10	Dizin belirtmeden açma	Mevcut dizine açılmalı	✓ Başarılı

7. Sonuç

Bu projede tarsau adlı arşivleme programı başarıyla geliştirilmiştir. Program, ödevde belirtilen tüm gereksinimleri karşılamaktadır:

- Metin dosyalarını tek bir .sau arşiv dosyasında birleştirebilmektedir

- Arşiv dosyasını açarak dosyaları istenilen dizine çıkartabilmektedir
- Dosya izinleri (okuma, yazma, çalıştırma) arşivleme sırasında kaydedilip açma sırasında geri yüklenmektedir
- Hatalı giriş durumlarında uygun hata mesajları verilip program sorunsuz şekilde sonlanmaktadır
- Binary dosyalar reddedilmektedir
- En fazla 32 dosya ve 200 MB toplam boyut sınırı uygulanmaktadır
- Makefile ile derlenebilmektedir

Geliştirme süreci GitHub üzerinden yürütülmüş olup proje deposuna <https://github.com/YesilBaraN/tarsau> adresinden erişilebilir.